

Distribute Hash Table (DHT)

What is DHT?

- All about locating content
- How to speed up? → Assign particular nodes to hold particular content
- A **hash table** stores (key, value/object) pairs
 - Fast lookup: $O(1)$
 - Widely used in content distribution networks

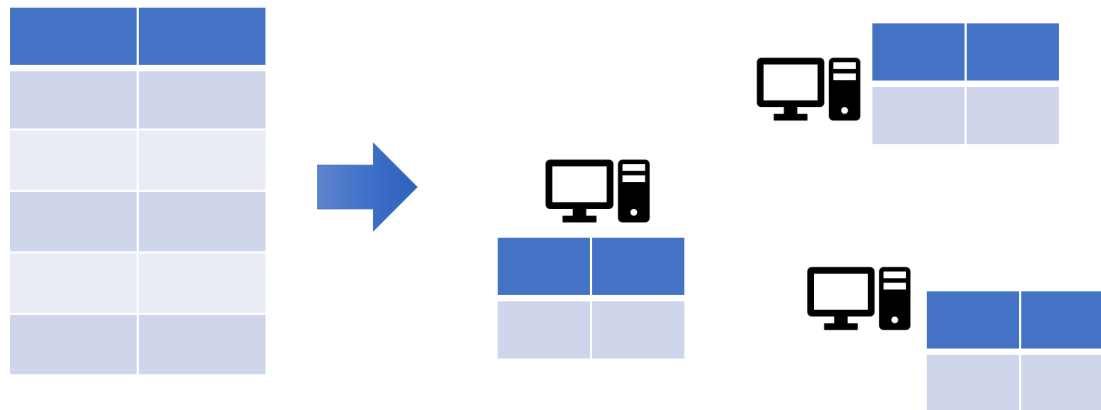
Key	Value
www.google.com	Page content
www.cnn.com	Page content
....	



What if the table is too large to be stored in a single computer?

What is DHT?

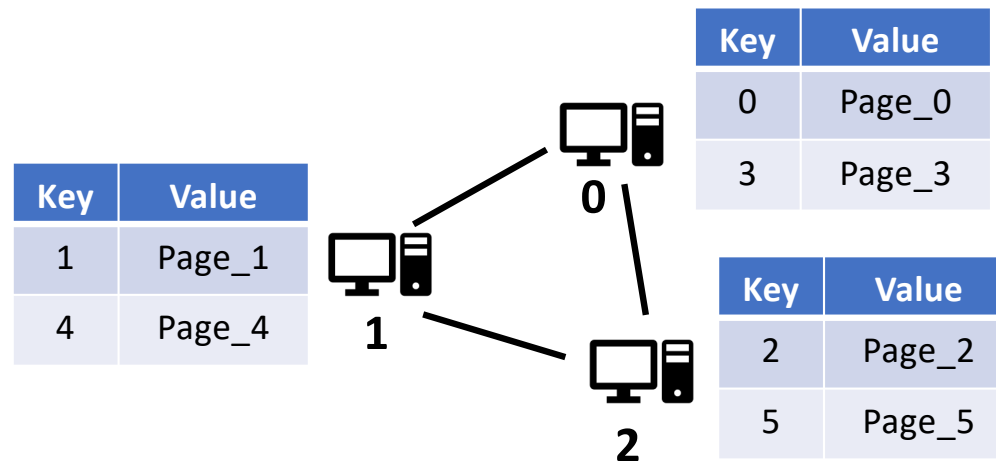
- Distributing a hash table to a network of nodes
 - Each node takes care of a subset of the table



- A virtual hash table
 - *Input*: key
 - *Output*: the node that stores the key

A Toy Example

- Consider 3 nodes, keys are integers
- Hash function: $\text{hash}(\text{key}) = \text{key} \% 3$



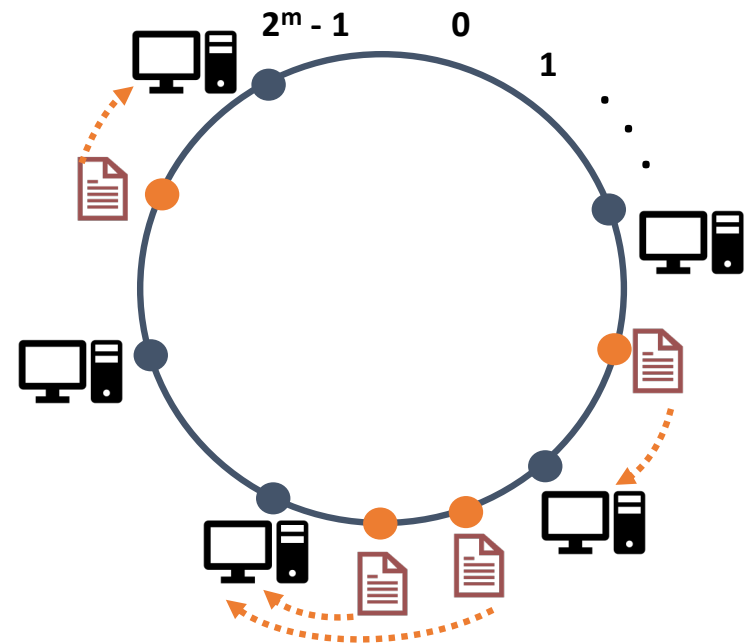
Problem: if one node leaves/joins, all items need to be rehashed

Consistent Hashing

- A special type of hashing: when the hash table is resized, the number of keys that need to be remapped is bounded
- Minimize the number of objects/items that need to be moved when a node leaves/joins the network
- All DHTs implement consistent hashing

Basic Idea

- Assume the hash function's output is a m -bit binary string $\rightarrow 2^m$ circular key space
- Hash both node ID and the key into the key space
- A key is stored at the node that appears next on the circle in clockwise order



Basic Components of DHT

- Overlapping key and node identifier space
 - Nodes and objects are all represented by m-bit binary string
- Building routing table
 - Next hops
 - Distance function (ID distance $\neq$ network distance)
 - Handle node joining and leaving
- Lookup and store interface

Comments

- A very popular research topic in networking and systems
 - Representative systems: Chord, CAN, Tapestry, Pastry...
- Used in practical systems such as BitTorrent, emule, blockchains, distributed file storage systems
- Nice system properties
 - Scalability, low latency, load balancing, dynamic configuration, resilience
- Not really used by Google searches or Facebook newsfeed, why?
 - Hard to handle intersection queries, sophisticated ranking, aggregation
 - Big companies prioritize centralized control with predictable latency and fast recovery, but they use DHT-like techniques internally

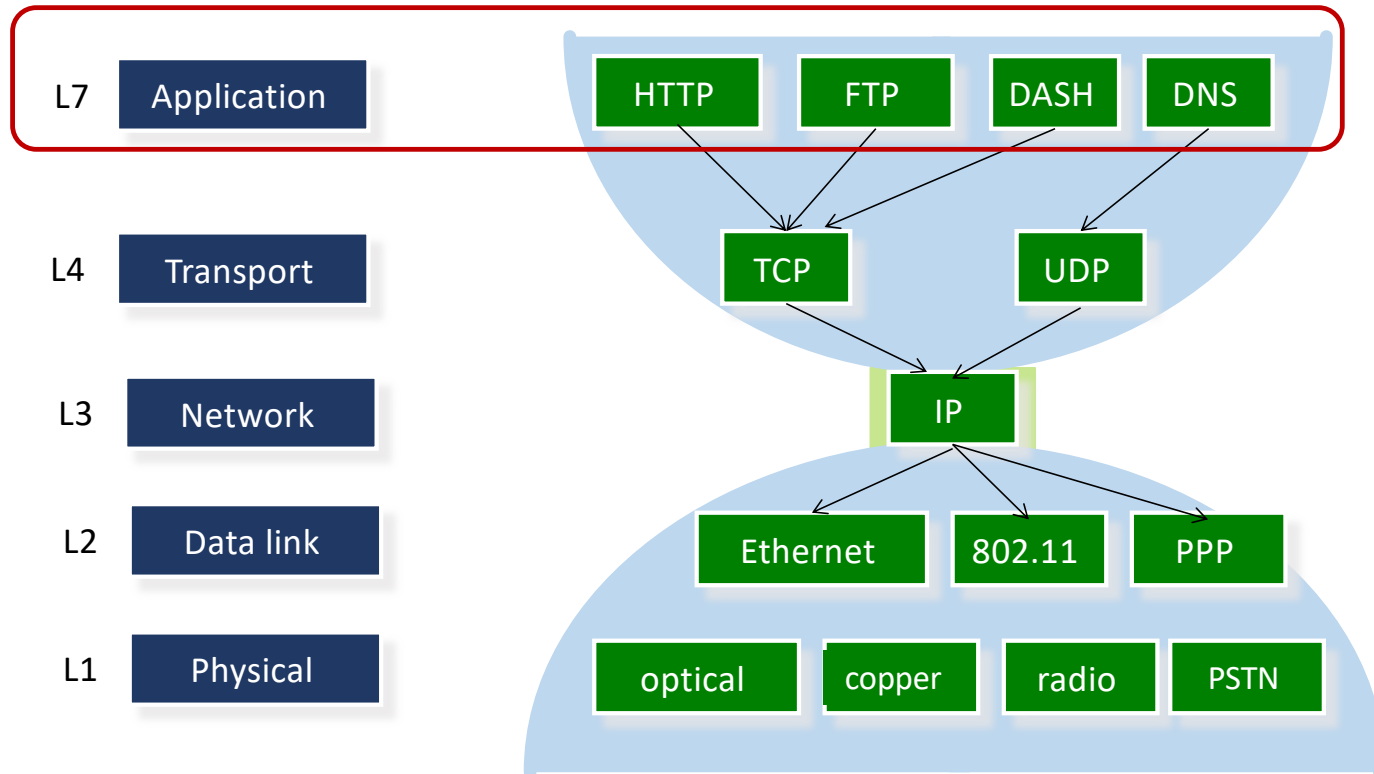


CSEE4119: Computer Networks

Transport Layer

Xia Zhou
Department of Computer Science
Columbia University

Slides credited to J.F Kurose and K.W. Ross and their textbook "Computer Networking: A Top-Down Approach"



Transport layer: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- Connectionless transport: UDP

Basics and UDP

- Principles of reliable data transfer
- Connection-oriented transport: TCP

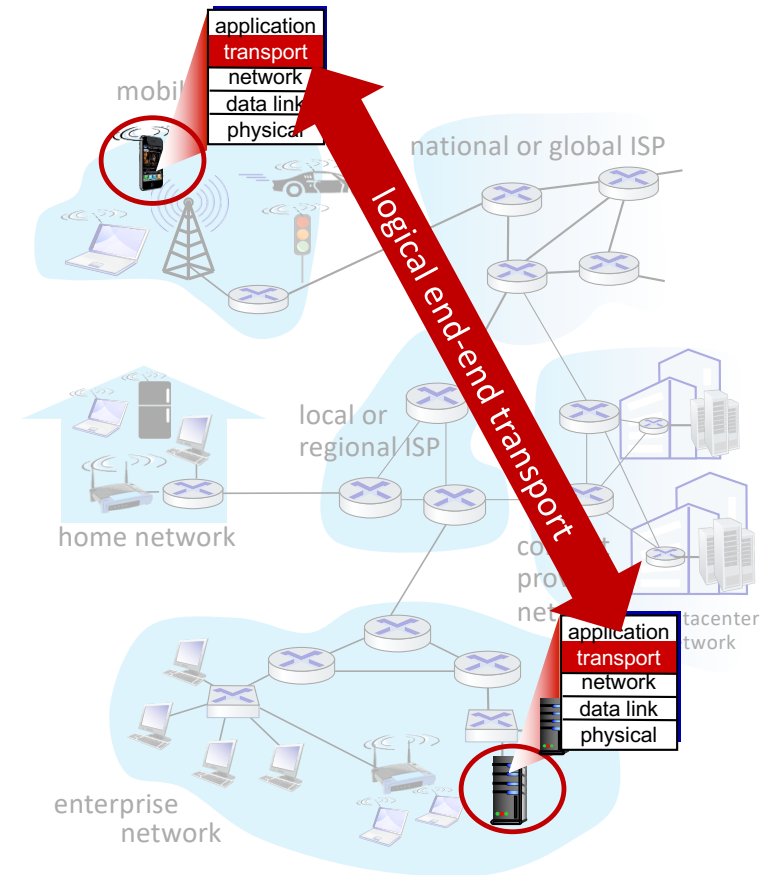
Reliability

- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

Congestion Control

Transport services and protocols

- provide *logical communication* between application processes running on different hosts
- transport protocols in end systems:
 - sender: breaks application messages into *segments*, passes to network layer
 - receiver: reassembles segments into messages, passes to application layer
- two transport protocols available to Internet applications
 - TCP, UDP



Transport vs. network layer services and protocols



household analogy:

12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes

Transport vs. network layer services and protocols

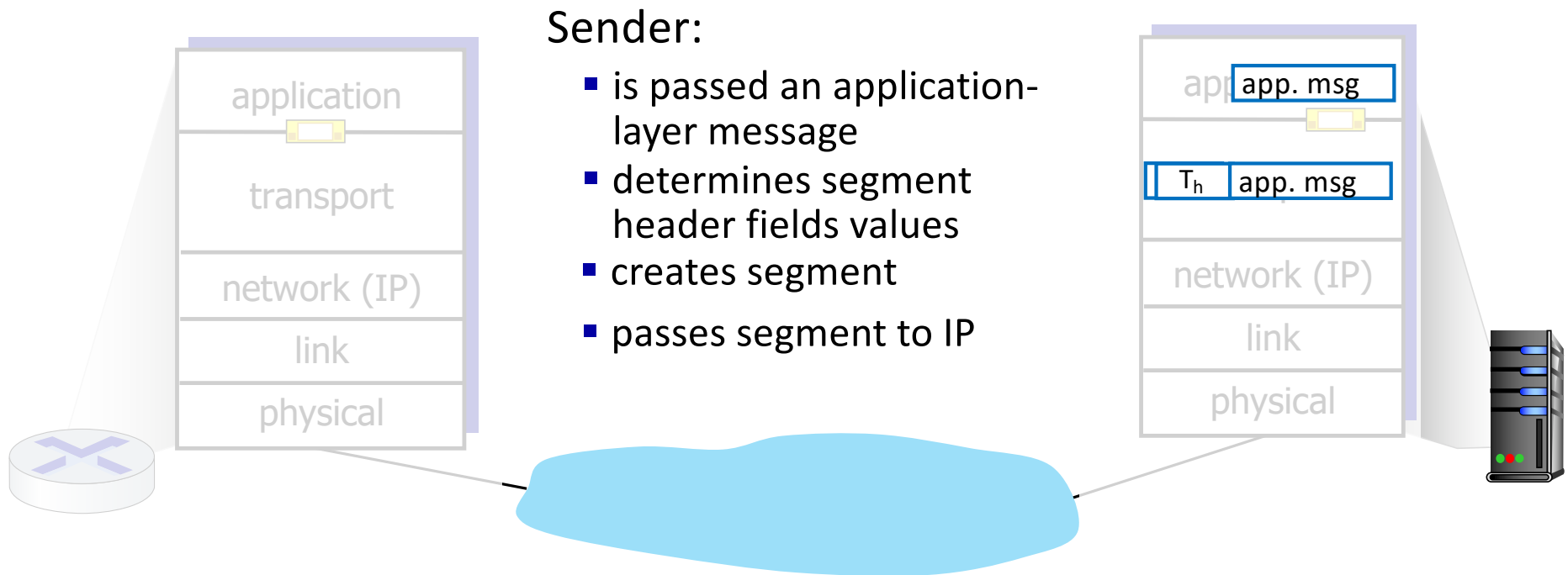
- **transport layer:**
communication between *processes*
 - relies on, enhances, network layer services
- **network layer:**
communication between *hosts*

household analogy:

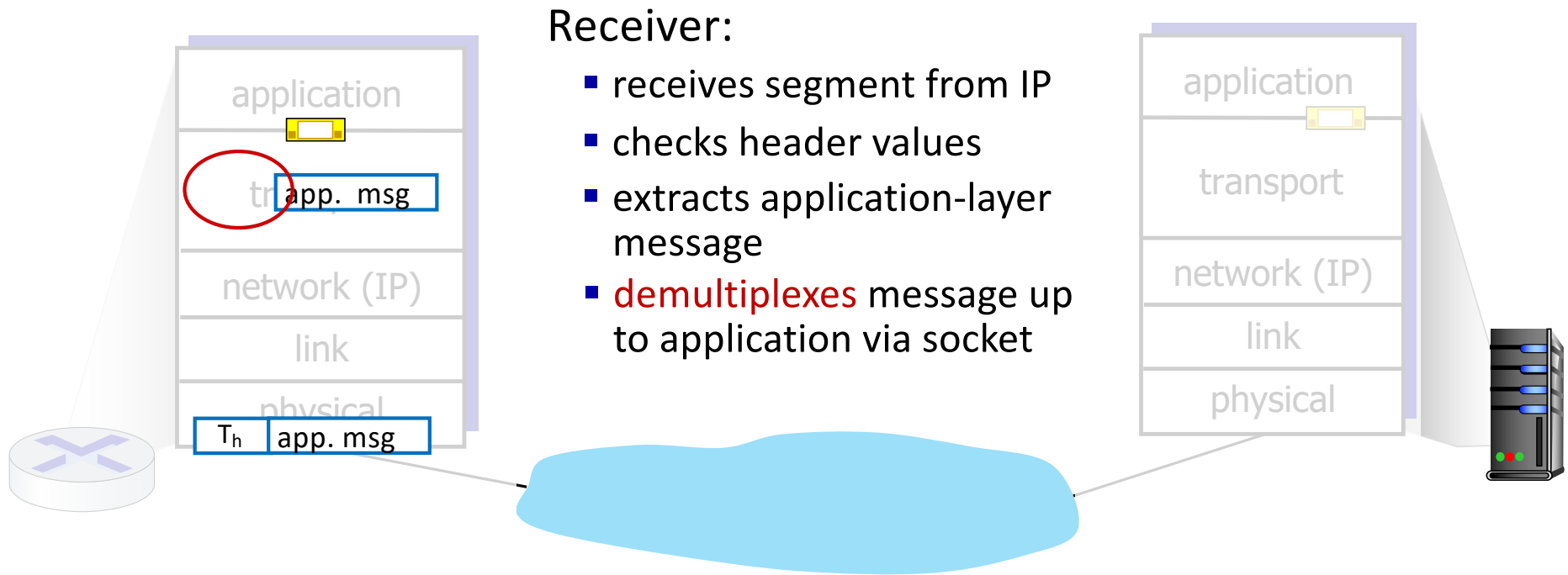
12 kids in Ann's house sending letters to 12 kids in Bill's house:

- hosts = houses
- processes = kids
- app messages = letters in envelopes

Transport Layer Actions

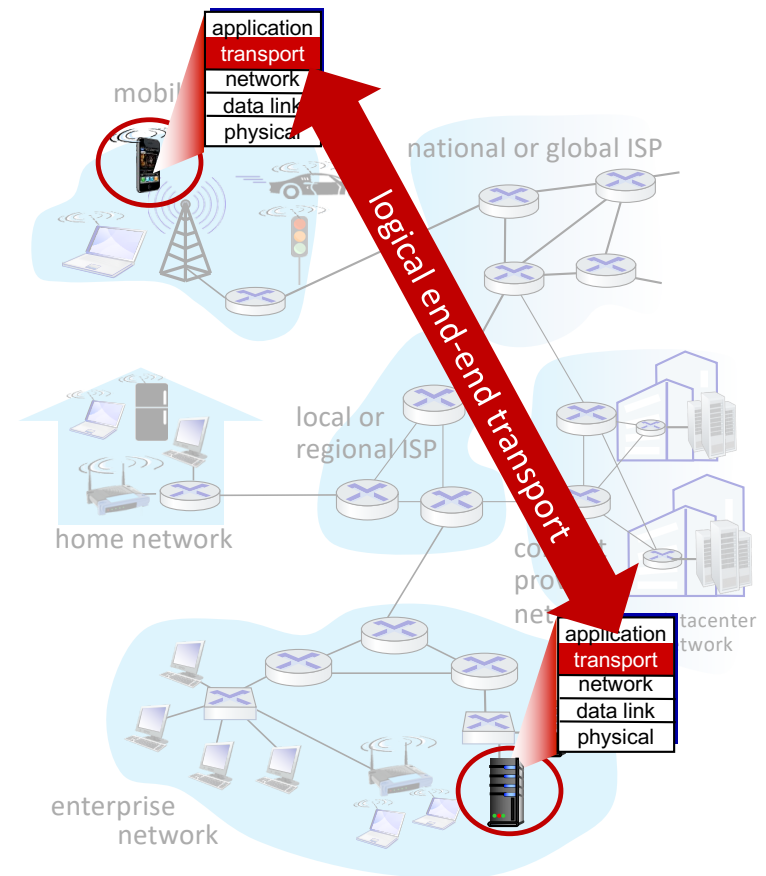


Transport Layer Actions



Two principal Internet transport protocols

- **TCP:** Transmission Control Protocol
 - reliable, in-order delivery
 - congestion control
 - flow control
 - connection setup
- **UDP:** User Datagram Protocol
 - unreliable, unordered delivery
 - no-frills extension of “best-effort” IP
- services *not* available:
 - delay guarantees
 - bandwidth guarantees



Transport layer: roadmap

- Transport-layer services
- **Multiplexing and demultiplexing**
- Connectionless transport: UDP
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality

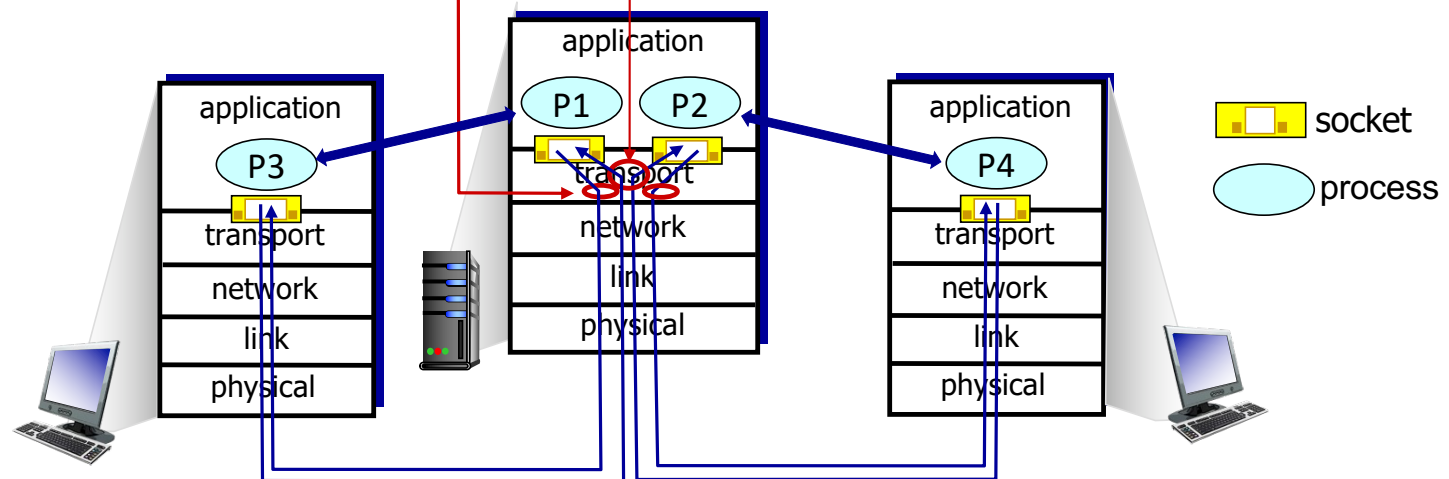
Multiplexing/demultiplexing

multiplexing as sender:

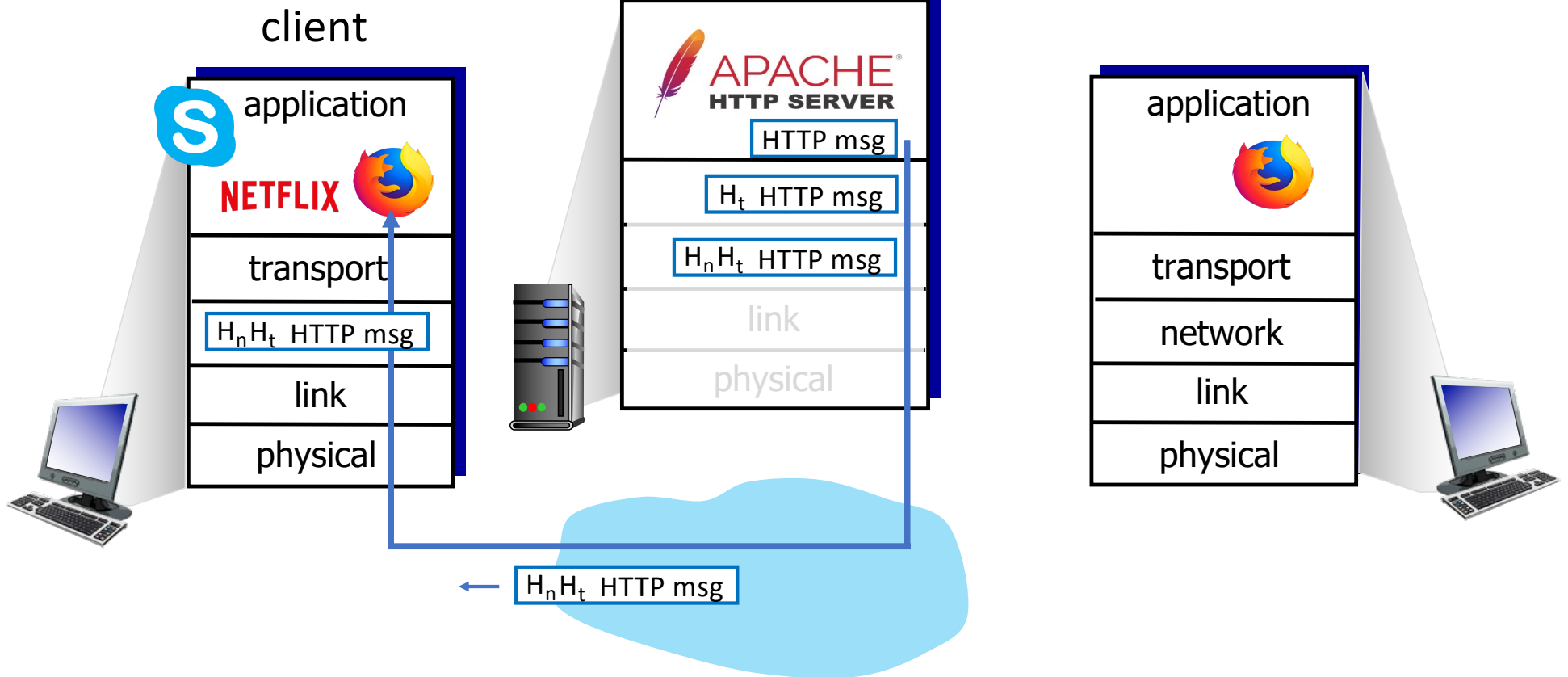
handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing as receiver:

use header info to deliver received segments to correct socket

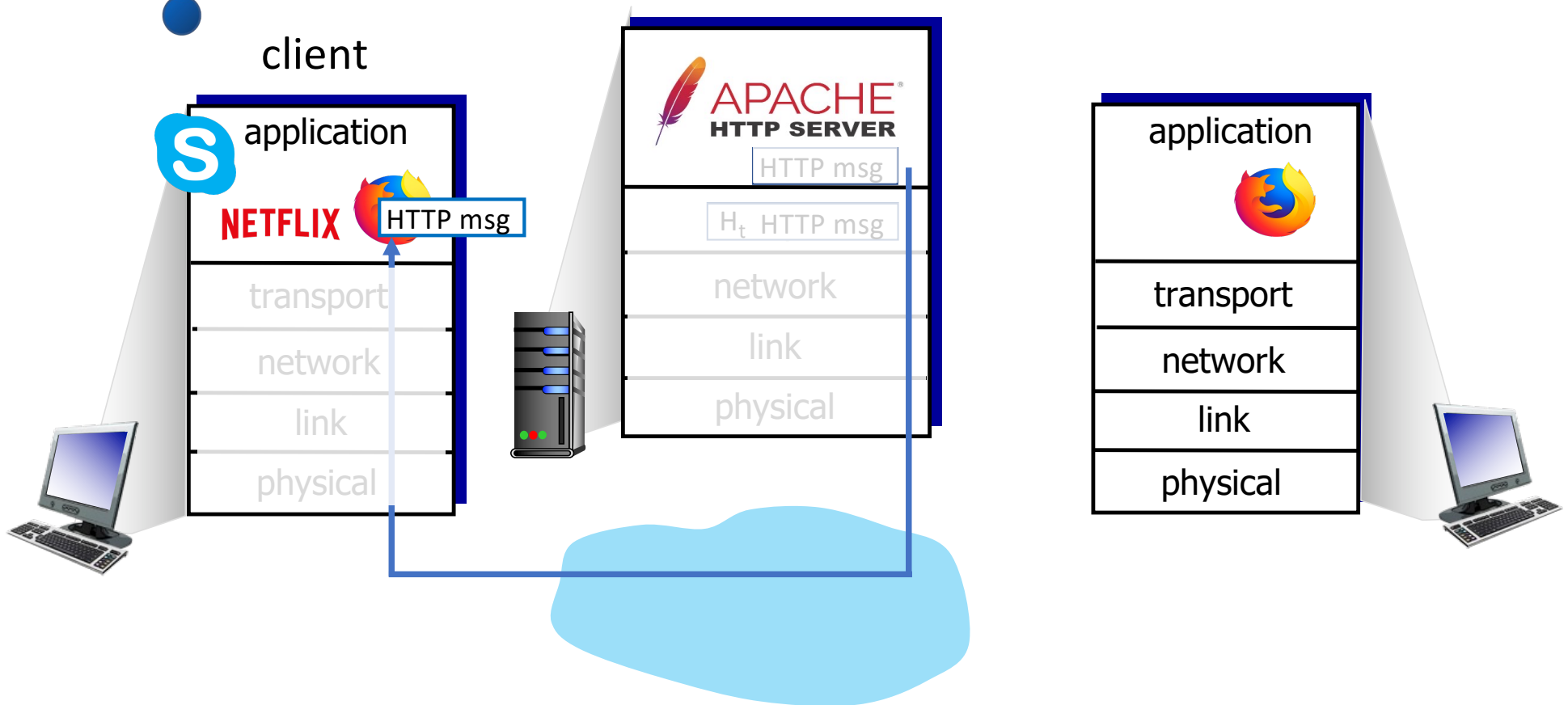


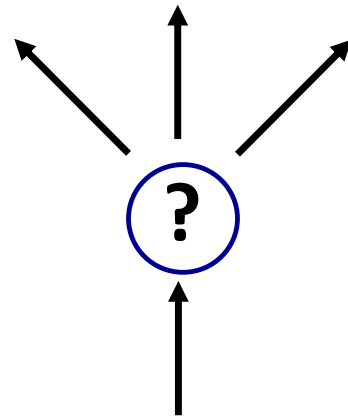
HTTP server



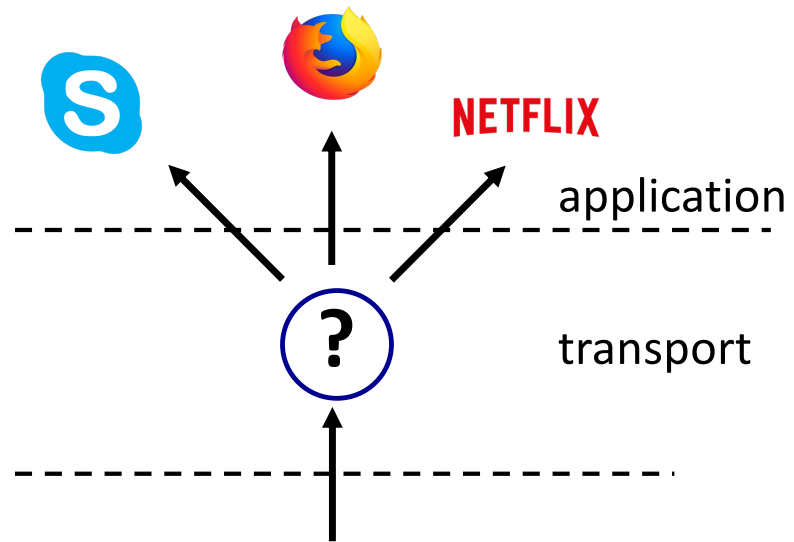


Q: how did transport layer know to deliver message to Firefox browser process rather than Netflix process or Skype process?





de-multiplexing



de-multiplexing





TSA Pre✓



Transportation
Security
Administration

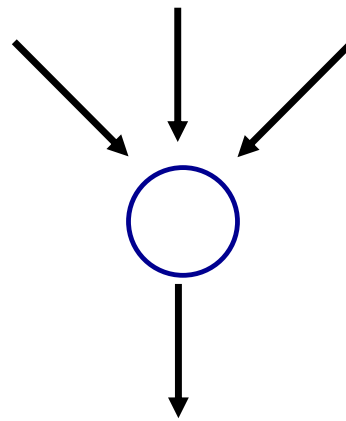
tsa.gov

Main
Checkpoint

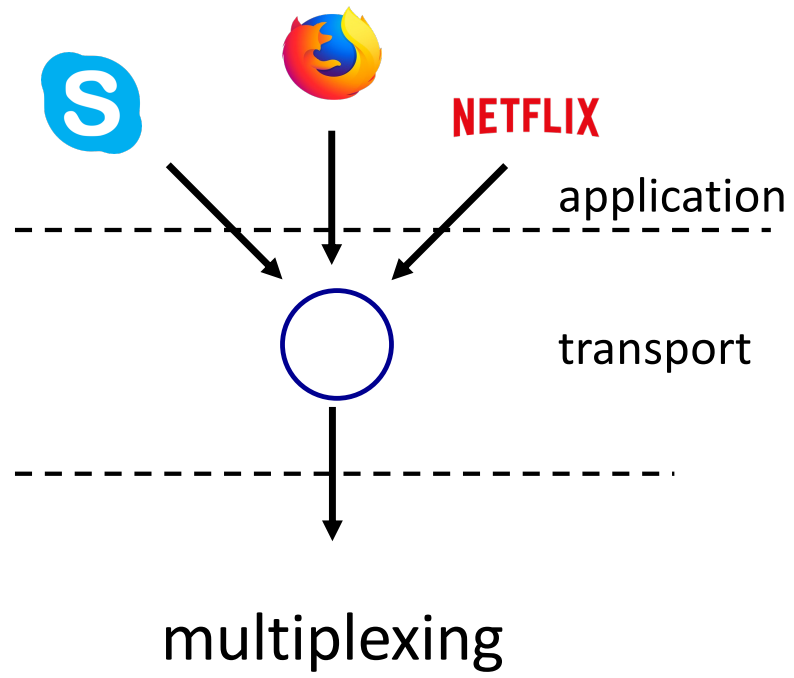


Transportation
Security
Administration

tsa.gov



multiplexing

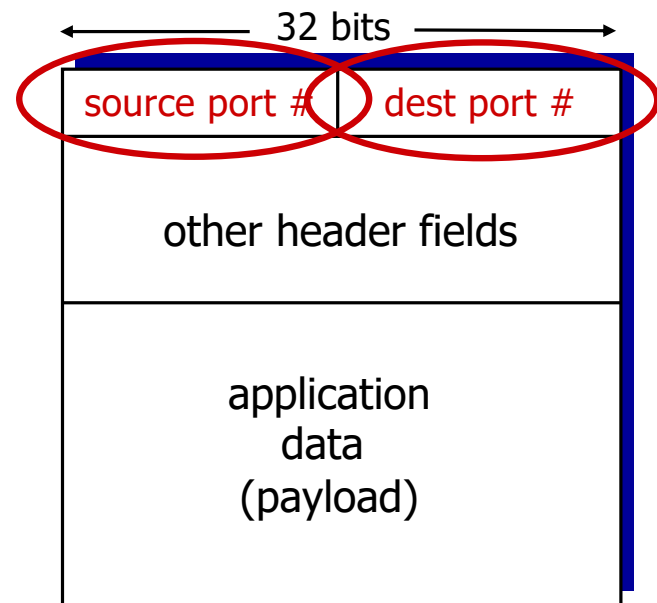




Multiplexing

How demultiplexing works

- host receives IP datagrams
 - each datagram has source IP address, destination IP address
 - each datagram carries one transport-layer segment
 - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket



TCP/UDP segment format

Connectionless demultiplexing

Recall:

- when creating socket, must specify *host-local* port #:

```
DatagramSocket mySocket1  
= new DatagramSocket(12534);
```

- when creating datagram to send into UDP socket, must specify
 - destination IP address
 - destination port #

when receiving host receives *UDP* segment:

- checks destination port # in segment
- directs UDP segment to socket with that port #



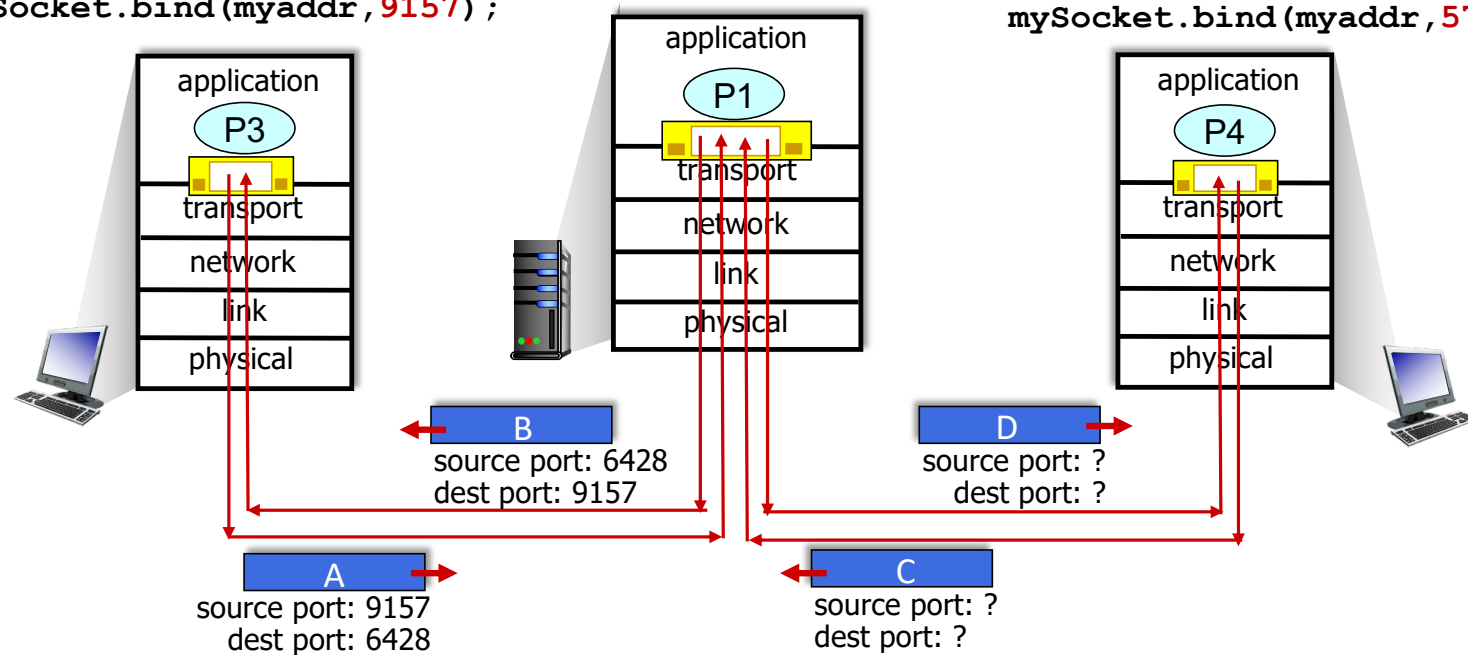
IP/UDP datagrams with *same dest. port #*, but different source IP addresses and/or source port numbers will be directed to *same socket* at receiving host

Connectionless demultiplexing: an example

```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 6428);
```

```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 9157);
```

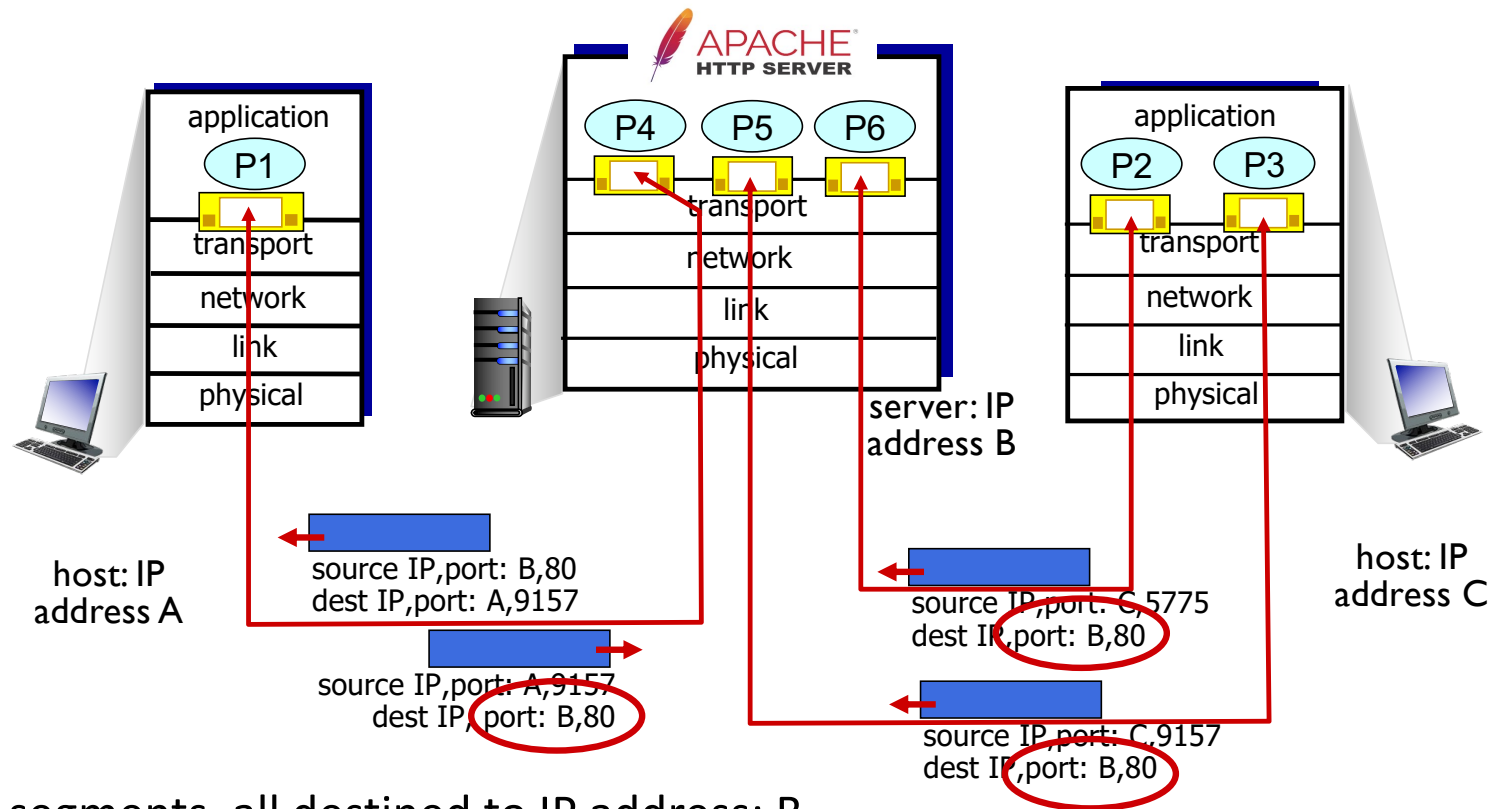
```
mySocket =  
    socket(AF_INET, SOCK_DGRAM)  
mySocket.bind(myaddr, 5775);
```



Connection-oriented demultiplexing

- TCP socket identified by **4-tuple**:
 - source IP address
 - source port number
 - dest IP address
 - dest port number
- demux: receiver uses *all four values (4-tuple)* to direct segment to appropriate socket
- server may support many simultaneous TCP sockets:
 - each socket identified by its own 4-tuple
 - each socket associated with a different connecting client

Connection-oriented demultiplexing: example



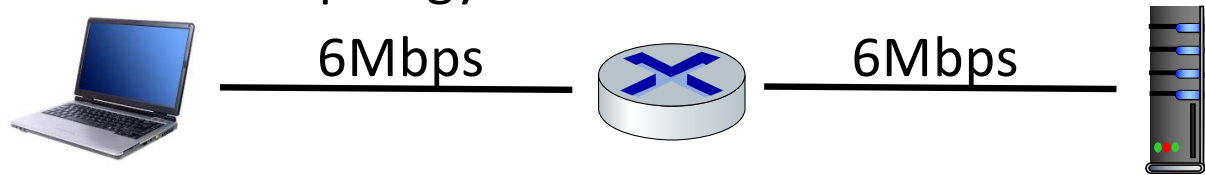
Three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

Summary

- Multiplexing, demultiplexing: based on segment, datagram header field values
- **UDP:** demultiplexing using destination port number (only)
- **TCP:** demultiplexing using 4-tuple: source and destination IP addresses, and port numbers
- Multiplexing/demultiplexing happen at *all* layers

Quiz

Your host are sending a message of 10 packets of 0.012Mb each to a server. See the topology and link bandwidth numbers as below:



We assume a negligible latency or propagation delay. The router uses store-and-forward transmission. How long would it take for the server to receive all 10 packets?